DataForce — build report (Milestone 1 + M1b)

Status:

- **Milestone 1** ("jalon 1": data, CRM and statistics engine) is complete and ratified.
- M1b (17 October, afternoon) adds:
  - a local web interface for testing the CRM;
  - "Connect TamTam" over MCP;
  - a 350-deal demo.

Every part was built, independently reviewed for security, fixed and re-verified. **712 automated tests pass.**

Date: 17 October 2026

Sources of truth:

- `docs/DECISIONS.md`: every decision and its reason.
- `docs/ARCHITECTURE.md`: the M1 contract.
- `docs/UI-CONTRACT.md`: the M1b contract.
- This PDF is generated from `docs/BUILD-REPORT.md` by `docs/tools/build_pdf.py`.

1. In one page

What DataForce is

An analysis workspace for sales and marketing teams. Instead of exporting their CRM into spreadsheets, a team keeps a *minimal CRM* in DataForce and asks it questions:

- *"Do inbound leads close better?"*
- *"Are enterprise deals slower?"*
- *"Who reports to whom at this account?"*

The CRM holds companies, people, deals and notes. TamTam already has better external data than we could build, so DataForce works one level down, at the CRM level. TamTam is *connected*, for contact details, only when a user explicitly asks.

The one rule that shapes everything

DataForce shows a number as a fact only when the evidence is strong enough. Otherwise it says **"not verified"** and gives the reasons:

- too few deals;
- an interval that is too wide;
- not significant once many comparisons are accounted for.

The AI never computes numbers. It may only explain numbers the code computed.

What is delivered

Piece	What it does	Where
Minimal CRM	Workspaces, team members, companies, contacts with "reports to", deals, append-only notes and comments, one-click erasure of a person or company	database + `app/services/`
Statistics engine	Win rate, sales-cycle length and deal size, with 95 % confidence intervals and "not verified" gates. A scan compares every group with the rest and controls false discoveries	`app/stats/`
Synthetic demo data	**350** realistic deals with effects planted on purpose, a pure-noise twin, and an answer key	`app/demo/`
Proof notebook	Runs everything end to end against the answer key	`notebooks/M1_engine_demo.ipynb`
Command line	`dataforce demo / stats / crm / db / ui / secrets`; every read command can answer in JSON	`app/cli/`
Web interface (M1b)	Accounts, contacts, org chart, deals, notes and comments, analysis page, settings. Runs on this computer only	`frontend/` + `app/api/`
Connect TamTam (M1b)	One click and a TamTam sign-in. The token is kept encrypted; the real tool list is shown; paid lookups only after a confirmed click	`app/connectors/`
LLM layer	Gemini (main) or Anthropic, with fakes by default. Used from Milestone 2	`app/llm/`

What it proves, on the demo with 350 deals (seed 42)

- **It finds what is there.** All 5 planted effects are verified, plus 4 real side effects of them.
- **It isn't fooled by small groups.** The "referral" trap stays "not verified": 25 deals with a 72 % win rate.
- **It invents nothing on that dataset.** There are 0 false findings.

On 100 fresh random datasets it never missed a planted effect. The price of more data is honesty about chance:

- about **1 in 12** planted datasets also shows one spurious finding;
- about **1 in 25** pure-noise datasets shows one.

That is the 5 % false-discovery rate doing what it promises. The noise twin at seed 42 happens to be one of them, and we kept it as an honest example (section 5).

Not delivered yet (by plan)

- the MCP server and the "AI-friendly" API for outside agents;

- login and hosting (M3);
- title normalisation, the inferred org chart and meeting-transcript themes (M2).

2. How it was built

The pivot.

- The project started (v1) as a "lookalike" tool that crawled company websites. On 17 October the product owner pivoted to DataForce, because TamTam already covers that ground.
- The v1 code that served only the lookalike idea was **deleted**.
- The infrastructure, the hardened configuration, the services pattern, the LLM layer and the TamTam MCP client were kept.

The team. A tech lead (Claude Opus) wrote the contracts first:

- the database model;
- the shared data types and the configuration;
- the API shapes;
- a written specification for each part.

Each part then went to its own builder agent (Claude Sonnet, low effort, as requested). File ownership did not overlap, so the agents could work in parallel. Separate reviewer agents could read but not change the code. They had to prove each finding, and fixers then closed them with regression tests.

Round	Build	Ratify (read-only)	Fix
M1	foundation, generator, stats engine, LLM/TamTam clean-up, integration	security SR4 and SR5, independent statistics review	3 fixers
M1b	API and CRM routes, TamTam connector, frontend, 350-deal demo	security SR6 (API), SR7 (OAuth), SR8 (frontend)	2 fixers + tech lead (frontend)

The tech lead checked every hand-back by re-running lint and tests rather than trusting the report, opened every screen in a headless browser, and kept `docs/DECISIONS.md` current.

Version control. The work is committed in logical steps on `main` (tag `m1` marks Milestone 1). It is local only: nothing is pushed.

3. Product decisions (and why)

Decision	Why
Not enough evidence → "not verified", always with the reasons, and "not verified" is explicitly *not* "no effect".	A salesperson must be able to rely on whatever is shown as a fact.
The AI never calculates. Numbers come from code (SQL counts, statistics); the AI may only name, explain and cite them.	Numbers must be reproducible and auditable.
Notes and comments are append-only. A correction is a new line pointing at the one it corrects; a delete hides the note but keeps the history. A database trigger enforces it.	Trust and traceability of what was said about a person or account.
Except GDPR erasure. Erasing a person or company hard-deletes them and everything attached.	The right to erasure takes precedence over history.
Strict minimum on third parties: a contact is name, title, company, source. An email or mobile exists only after an explicit, confirmed TamTam lookup. It is stored apart, encrypted, with its own *Delete* button.	Data minimisation (GDPR); no surprise personal data.
Every table belongs to a workspace.	One team = one workspace; demo datasets live side by side; opening the tool to TamTam clients later needs no schema rewrite.
Synthetic data, announced as such (a permanent banner in the UI).	No real personal data during the build; a known answer key lets us *prove* the engine is right.
Paid TamTam enrichment only on an explicit click, after a dialog that states the credits. There is a per-call ceiling, and the server refuses if the price rose above what the user accepted.	No surprise spending.
Web interface pulled forward (M1b).	The product owner wants to judge the CRM scope by using it, not from a document.
TamTam connected from the interface over MCP, with TamTam's own sign-in page; nobody copies a token. One connection per workspace.	Simple for the user; the token never passes through a person or a chat.
Demo workspaces never call TamTam.	Their people are fictional: spending real credits on them would be waste.
Local only, no login yet.	Login and hosting are M3; until then the app refuses to listen anywhere but this computer.
350 demo deals (was 200).	Product-owner choice: no planted effect is missed any more (section 5).

4. How the statistics work (plain language)

The three questions.

- *Win rate*: of the deals that are closed, what share was won?

- *Sales cycle*: for closed deals, how many days from creation to close (median)?
- *Deal size*: the median amount in euros.

Medians are used because one giant deal should not distort the picture.

Confidence intervals. Every number comes with a range that, 95 times out of 100, contains the true value. With 20 deals the range is wide; with 300 it is narrow.

- *Wilson* for a rate, reliable even with few deals.
- *Newcombe* for the difference between two rates.
- *Bootstrap* for medians: the deals are re-sampled 2,000 times, with a fixed seed so results are reproducible.

The "not verified" gates (approved by the product owner, all configurable):

A result is "not verified" when…	Default
the group has too few deals	fewer than 30
a win rate rests on too few wins or losses	fewer than 10 of either
the range is too wide	more than ±15 points for a rate, more than ±30 % of the value for a median

Comparing groups.

- The scan compares each group with *the rest* (e.g. inbound leads against all other sources) on each question.
- It uses Fisher's exact test for rates and the Mann-Whitney test for days and amounts.
- A scan makes about 90 comparisons at once, so some would look "significant" by pure chance. The *Benjamini-Hochberg* correction keeps the expected share of false findings among the reported ones at 5 %.
- A finding is verified only if:
  - both sides pass the gates;
  - the corrected test is significant;
  - the range of the difference excludes zero.

Missing data is counted and reported, never turned into a group of its own.

What this means in practice. These are real lines from the demo scan (350 deals, seed 42, 263 of them closed), as the analysis page shows them:

Group vs rest	Question	Result	Verdict and reason
Inbound leads	win rate	72.5 % vs 24.2 %, difference +48.4 pts [+35.9, +58.6]	**verified**
Enterprise	sales cycle	median 106 vs 41 days, +66 [+53, +74]	**verified**
Mid-market	sales cycle	median 60 vs 41 days, +19 [+6, +25]	**verified** (q = 0.034)
Referral leads	win rate	72.0 % vs 39.1 %	not verified: only 25 deals (< 30), only 7 losses

Mid-market	deal size	median €20,771 vs €15,879	not verified: not significant (q = 0.58), range [−190, +9,921] EUR includes 0

The referral line shows the rule at work. The gap looks spectacular, but it rests on 25 deals and 7 losses. So DataForce says "not verified" rather than put a fragile number in front of a salesperson. "Not verified" never means "no effect". It means "not enough evidence yet".

5. The synthetic demo data

What the generator builds. A realistic small company:

- 122 accounts;
- 755 contacts with full org charts (French and English titles, some messy or missing);
- 350 deals across owners and stages;
- notes and comments, including corrections.

Everything is invented (`.example` domains, made-up names) and deterministic: the same seed always gives the same data.

Kind	In the demo data
Planted on purpose (the engine must find it)	Inbound leads win far more often than the rest. Enterprise deals take much longer to close than SMB deals, and are much larger.
Planted as a trap (must stay "not verified")	Referral leads: a very high win rate, but never more than 25 closed deals.
Null (must find nothing)	Industry, region and owner have no effect anywhere. The whole **noise variant** has the same shape and no effect at all.

Side effects are expected. Some differences are real consequences of the planted ones without being planted themselves:

- small deals close faster, because most of them are SMB deals;
- outbound wins less, because inbound wins more.

The answer-key checker lists these separately and never counts them as errors. It counts as a **false finding** any verified result:

- in a cell with no planted link;
- on the small trap group;
- in the wrong direction.

How reliable is it? 100 random datasets for each variant:

Measure	350 deals (now)	200 deals (before)
Closed deals per dataset	about 254	about 165
Each planted effect found	**100 %**	93–100 % (inbound was the weak one)

Planted datasets with a spurious finding	8 %	0–1 %
Noise datasets with any verified finding	4 %	1 %

How to read it. More data removed every miss. With more real effects verified per scan, the 5 % false-discovery control lets through an occasional chance finding, and the test is deliberately strict: any such finding fails the answer-key check. The engine can be wrong in both directions, and it never claims otherwise.

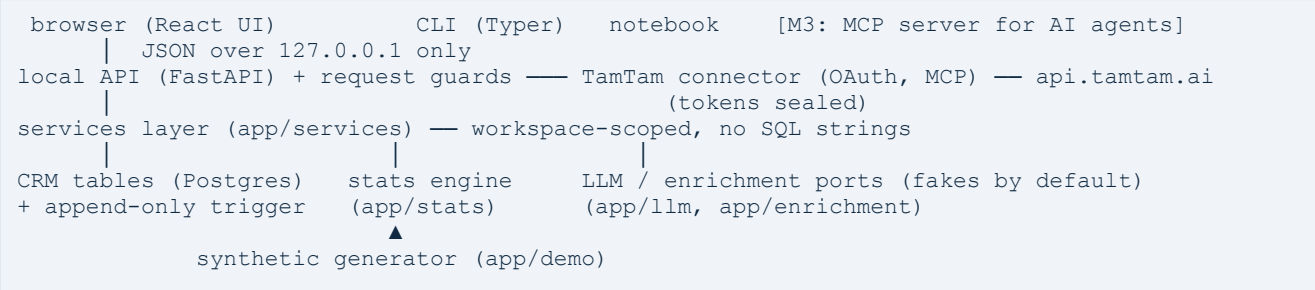
At seed 42 the noise twin shows one chance finding: one sales rep's sales cycle, q = 0.028. We kept that seed rather than pick a luckier one. The check reports it as a false positive, and a test pins it.

6. Architecture and structure

Layers. Every way into the system goes through one *services* layer. That layer is the only code that talks to the database. The ways in are:

- the command line;
- the notebook;
- since M1b, the local web API and the interface.

Analyses are pure functions that receive plain rows and return plain JSON-ready results.



Folder structure. About 9,900 lines of backend code, 580 lines of migrations, 8,000 lines of tests and 1,900 lines of frontend.

Folder	Contents
`backend/app/domain/`	The contracts: enumerations, shared data types, provider interfaces
`backend/app/db/`	Database models (12 tables) and the hardened session
`backend/alembic/versions/`	Migrations: schema and append-only trigger, lock-down of public API roles, hardening, connector tables
`backend/app/services/`	CRM operations, enriched channels, runs history, analytics, admin
`backend/app/stats/`	Intervals, significance tests, gates, the scan engine
`backend/app/demo/`	Generator, answer-key checker, database loader
`backend/app/api/`	Local HTTP API: routes, request guards, error mapping, hosting of the built interface

`backend/app/connectors/`	TamTam OAuth client (discovery, registration, PKCE, refresh), sealing, credential store
`backend/app/llm/`, `app/enrichment/`	AI providers and the untrusted-text guard; the TamTam MCP client (tool allow-list, credit guard) and the fake
`backend/app/cli/`	The `dataforce` command
`frontend/`	React interface (Vite, TypeScript), no UI kit
`docs/`	Decisions, contracts, security reviews, statistics ratification, this report

Data model.

Table	Holds	Notes
`workspace`	one team's data, or one demo dataset	everything below belongs to a workspace
`app_user`	sales-team members (deal owners, note authors)	login arrives in M3; the UI has an "acting as" picker meanwhile
`account`	companies	industry, segment, region, size, source
`contact`	people at a company	name, title, "reports to", source; no email or phone
`deal`	opportunities	amount, stage, lead source, created and closed dates, owner
`note` / `comment`	free text on a person or a company	append-only, corrections chain, soft delete
`run`	history of generations and scans, with the full result	lets any number be re-read and cited
`llm_call`	one line per AI call	usage and cost per workspace
`connector_credential`	a workspace's TamTam connection	tokens sealed; never sent to the browser
`oauth_pending`	a sign-in in progress	single use, 10 minutes, only hashes of the secrets
`contact_channel`	an email or mobile found by a confirmed lookup	value sealed; one-click delete; erased with the contact

Technology.

Choice	Why
Python 3.13, SQLAlchemy 2 (sync), psycopg 3	Mature and simple; async only where there is network I/O
PostgreSQL 16 + pgvector (Docker locally, pinned image)	One database for records, exact aggregates and later text search
numpy + scipy	Reference implementations of the statistical tests

pydantic 2	Every result is a typed, JSON-serialisable object, the same for the CLI, the API and later AI agents
FastAPI + uvicorn (M1b)	Typed HTTP API with a machine-readable contract (`/api/openapi.json`)
React 19 + Vite + TypeScript, no UI kit (M1b)	Small, fast and dependency-light: 93 npm packages, all pinned, install scripts disabled
OAuth 2.1 with PKCE, dynamic registration, resource indicator (M1b)	What TamTam's MCP server publishes; the standard way MCP clients connect
Fernet encryption (`cryptography`) (M1b)	Tokens and enriched values are unreadable in the database and in backups without the key
Gemini (main) / Anthropic (swap), behind one interface	Product-owner choice; fakes keep tests and demos free and offline
Hashed lock files (pip-tools, `package-lock.json`)	Reproducible, tamper-evident installs

7. Security

Principles.

- Secrets live only in the environment or sealed in the database. They are never logged, never shown in errors and never sent to the browser.
- Data on third parties is kept to the least possible.
- Untrusted text cannot act as instructions to the AI.
- The database itself refuses what the application must never do.
- The web interface listens only on this computer. Another website the user visits cannot drive it.

Reviews. Reviewer agents could read the code but not change it, and had to prove each finding with a reproduction.

Review	Scope	Findings	Status
SR1–SR3 (v1)	Crawler; LLM and TamTam client; database, configuration, supply chain	—	closed, re-verified in SR4 and SR5
SR4	CRM database, append-only trigger, workspace isolation, erasure, infrastructure	0 critical, 0 high, 3 medium, 5 low, 4 info	all medium fixed; the rest fixed or accepted with a reason
SR5	LLM / TamTam re-check, statistics and demo data integrity	0 critical, 0 high, 4 medium, 6 low, 3 info	all medium fixed; the rest fixed, accepted or moved to M2
SR6 (M1b)	Local API: forged requests from other websites, DNS rebinding, cross-workspace access, injection, leaks	0 critical, 0 high, 0 medium, 5 low, 5 info	all 5 low fixed with tests

SR7 (M1b)	TamTam sign-in, secrets, paid lookups	0 critical, 0 high, 0 medium, 4 low, 4 info	all 4 low fixed with tests; 3 info fixed, key rotation accepted (documented)
SR8 (M1b)	Frontend and npm supply chain	0 critical, 0 high, 0 medium, 2 low, 2 info	both low fixed with tests

Main things the reviews tested and confirmed hold:

- Notes cannot be rewritten or hard-deleted, even by direct SQL.
- Erasing a person or company leaves no row behind, including their stored email or mobile.
- No route returns or changes another workspace's data, for any identifier tried.
- Cross-site attacks on the local app fail.
  - A forged request from another website is refused (Origin check plus a custom header, with no CORS).
  - A DNS-rebinding hostname is refused.
  - Form posts are refused.
  - The strict content-security policy blocks injected scripts.
- The TamTam sign-in.
  - It cannot be redirected to another server.
  - A stolen or replayed callback is useless: single-use state bound to the browser, PKCE, issuer check.
  - The token is only ever sent to TamTam's MCP address.
- Paid lookups.
  - A paid lookup cannot run without a literal `true` consent and a credit amount the user accepted.
  - It never runs in a demo workspace.
  - A second lookup for the same contact while one is running, or a paid lookup for a value already stored, is refused before anything is sent.
- **Server text.** Tool descriptions, notes and names coming from servers or imports are shown as plain text, never as HTML.

Accepted residuals (written down, scheduled):

- No login until M3. Any program already running on this computer can use the local app, and the "acting as" picker is a label, not an identity.
- The encryption key sits in `backend/.env`. Losing it means reconnecting TamTam and losing stored emails and mobiles; there is no key rotation yet.
- TamTam's tool names, credit prices and refresh behaviour are still assumptions until a real account is connected. The interface labels prices "assumed".
- Erasure does not scrub free-text mentions of a person in *other* notes or in past scan results (sweep planned with M2).
- Workspace boundaries are enforced in the services, not yet by composite database keys. The app should run under a non-owner database role when hosted (M3).

8. Verification results

Check	Result
Backend tests	**698 pass**: database, config, CRM services, statistics, demo data, LLM, TamTam client and connector, API, end-to-end command line. Live tests against real AI providers and TamTam are opt-in and were not run
Frontend tests	**14 pass** (org-chart tree, number formatting, sign-in address check, error parsing); type check and production build clean
Lint and format (ruff)	clean on the whole backend
Migrations vs models (`alembic check`)	no difference; dev database at `0004_connectors`
Running app	started with `make ui`. Every screen opened in a headless browser with no console error, and every guard was probed with real requests
Notebook	re-executed at 350 deals, saved with outputs, no secret or database address in the outputs
Statistics ratification	**RATIFIED** (M1): every method matches an independent re-implementation
Security	SR4–SR8 resolution tables (section 7)

Caught during the verification itself:

- **The 350-deal switch revealed the honest cost of more data.** Chance findings appeared at the rate the 5 % control allows. The end-to-end tests now pin noise seed 42 as a known false alarm instead of hiding it.
- **A reviewer found that opening the app as `localhost` instead of `127.0.0.1` would break the TamTam sign-in.** The cookie and the return address were on different hosts. It was fixed.
- **The tech lead found a route reading the wrong settings** when the app runs on a custom port, which would have broken the sign-in return address. It was fixed.
- **A not-verified reason printed an amount as "9.92e+03".** Amounts and days now read in plain units.

9. How to run it

Prerequisites: Docker, Python 3.13, Node 20+. Full instructions are in `README.md`.

```
make db-up                                    # local Postgres on 127.0.0.1:5433
make migrate                                  # schema (after the one-time Python setup)
backend/.venv/bin/dataforce demo generate --variant planted --seed 42 --replace
make ui                                       # then open http://127.0.0.1:8800
```

To try TamTam:

1. Create a real workspace (workspace menu, *+ New workspace…*).
2. Open *Settings* and click *Connect TamTam*.
3. Sign in on TamTam's page.
4. Click *Check connection* to see the tools TamTam really offers.

10. What comes next

Milestone	Content
M2	Title normalisation (FR/EN dictionary, AI only for unknown titles, cached and flagged); org-chart rules with their evidence; transcript import (PDF / Markdown) and meeting themes counted by code, named by the AI with verified quotes
M3	MCP server and API for AI agents (exact results), login, hosting, and the rest of the interface

If time runs short, cut meeting analysis first and keep the org chart, because it complements TamTam.

Open questions for the product owner.

1. **CRM scope.** You are testing the minimal CRM in the interface. Is it enough?
2. **TamTam.** After connecting, does *Check connection* show the tools we expect? What are your real credit prices? We still assume about 1 credit for an email and 10 for a mobile.
3. **AI key.** Add `GEMINI_API_KEY` to `backend/.env` yourself before M2. Never paste it in a chat.
4. **Noise seed 42.** We kept it, with its one chance finding, as an honest example. Prefer another seed for demos?
5. **GitHub.** Everything is committed locally. Should it be pushed to a (private) GitHub repository?

Glossary

Term	Meaning
Confidence interval (CI)	The range that most likely contains the true value; narrower with more data
Gate	A minimum-evidence rule; failing one makes a result "not verified"
FDR (false discovery rate)	The expected share of reported findings that are false; held at 5 %
Planted effect	A difference written into the synthetic data on purpose, so we can check the engine finds it
Null dimension	An attribute with no effect at all; the engine must not report one
Append-only	Text that can be corrected or hidden but never silently rewritten
Workspace	One team's isolated data
MCP	Model Context Protocol: the standard way AI agents call tools; TamTam exposes one, DataForce will too
OAuth / PKCE	The standard "sign in with…" flow, and its protection against a stolen sign-in code
Sealed	Encrypted with the app's secret key, unreadable without it
CSRF / DNS rebinding	Two ways a malicious website tries to make your browser talk to an app on your own computer; both are blocked